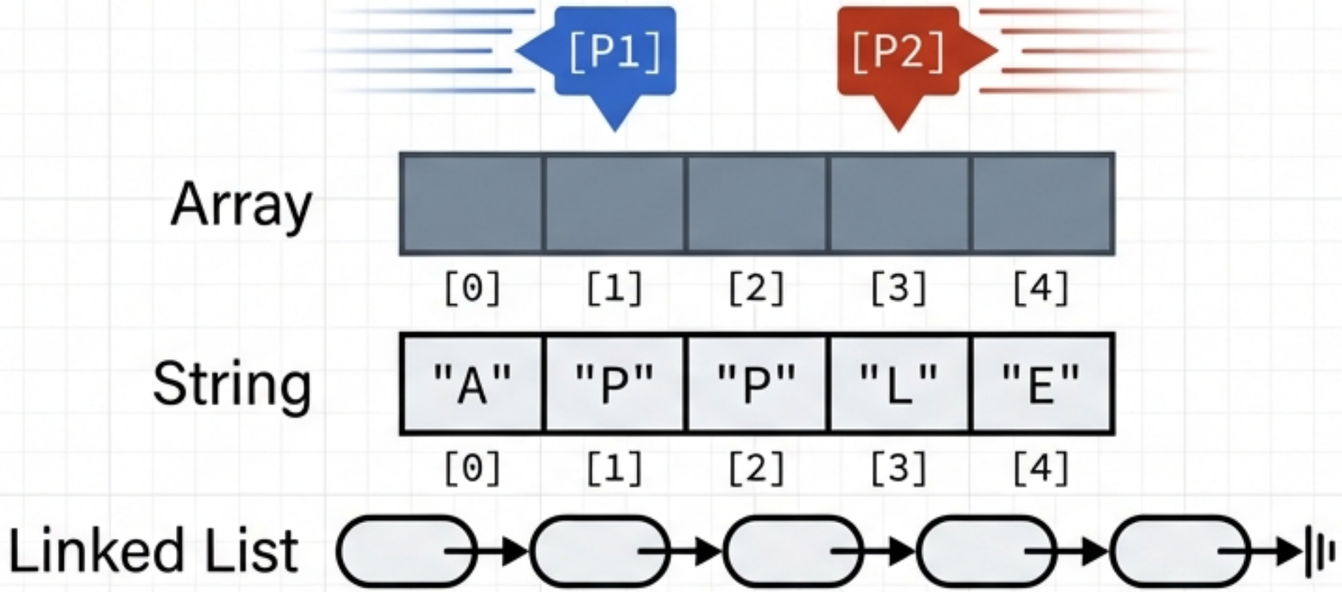
DSA Pattern #1 / Day 1 of 90

Algorithmic Playbook: The Two Pointers Pattern

Master in-place optimization, linear time execution, and algorithmic triggers.

Two Variables Traversing a Linear Structure



The Definition

An optimization technique using two variables (pointers/indices) to simultaneously traverse a linear data structure until a specific termination condition is met.

The Constraint

Exclusively applicable to sequential memory structures: arrays, strings, and linked lists.

The Goal

Eliminate nested loops. Process elements in a single, highly efficient sweep.

The PCS Diagnostic: When to Trigger the Pattern

Does the problem require...

[P] Partitioning

Trigger: Remove duplicates, move zeroes, or segregate elements.

[C] Comparisons

Trigger: Compare from both ends, palindrome checks, or reversing.

[S] Searching

Trigger: Find specific pairs, target sums, or sub-array boundaries.

If the problem description matches any of these three conditions on a linear data structure, default to the Two Pointers pattern.

The Three Primary Pointer Configurations

Opposite Ends (The Squeeze)

Used for: Sorted arrays, target sums, palindrome checks.

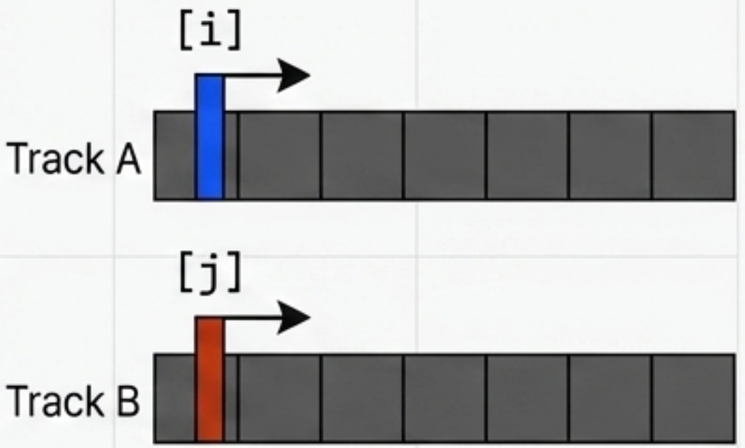
Termination: Ends when pointers cross or match.

Same Direction (Fast/Slow)

Used for: Cycle detection, finding Linked List middle, in-place removals.

Termination: Ends when the fast pointer hits null.

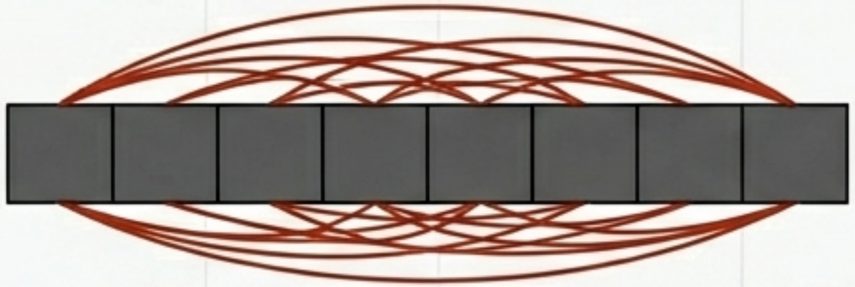
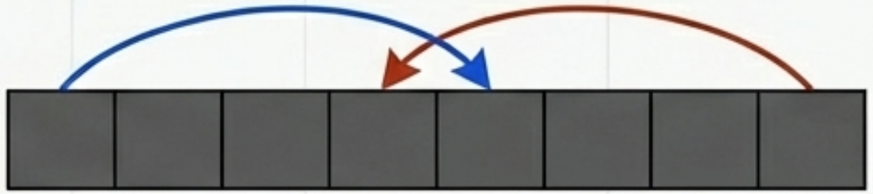
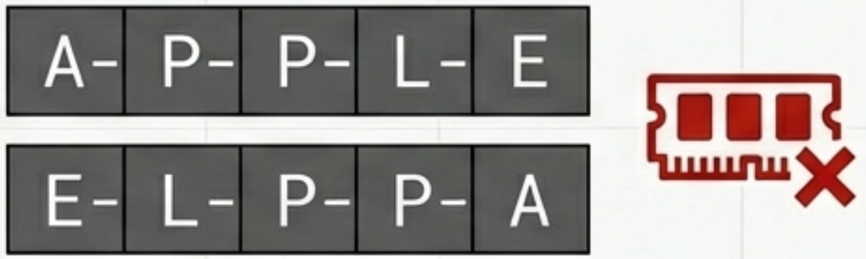
Two Inputs (The Merge)



Used for: Merging sorted arrays, finding intersections.

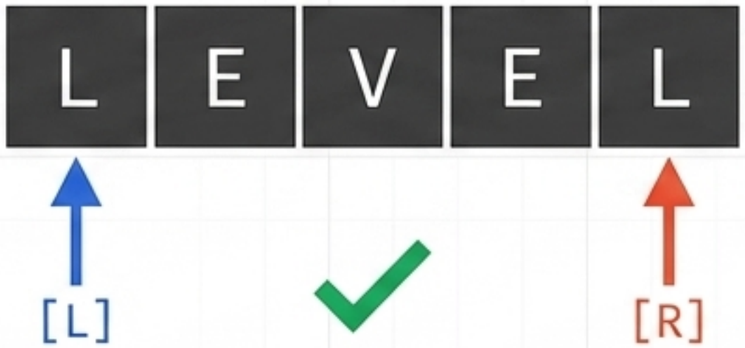
Termination: Ends when one or both arrays are exhausted.

The Efficiency Matrix: Unlocking O(N) and O(1)

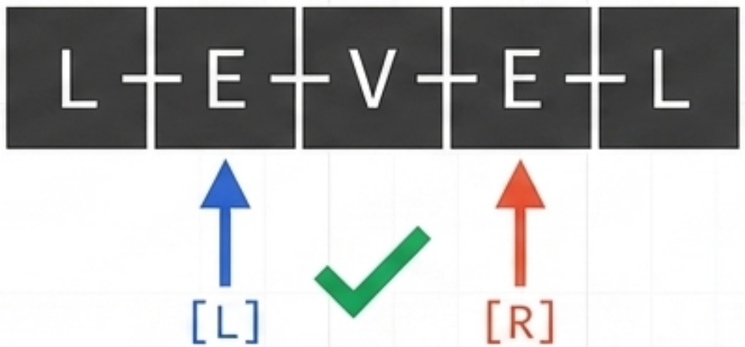
	Brute Force	Two Pointers Pattern
Time Complexity (Target Sum)	 Tests all N^2 pair combinations.	 Sweeps inward in linear O(N) time. Stops early on failures.
Space Complexity (Palindrome)	 Duplicates entire structure in memory O(N).	 Operates 'In-Place' using O(1) space. Zero new arrays created.

Playbook 1: Opposite Ends (The Squeeze)

1.



2.

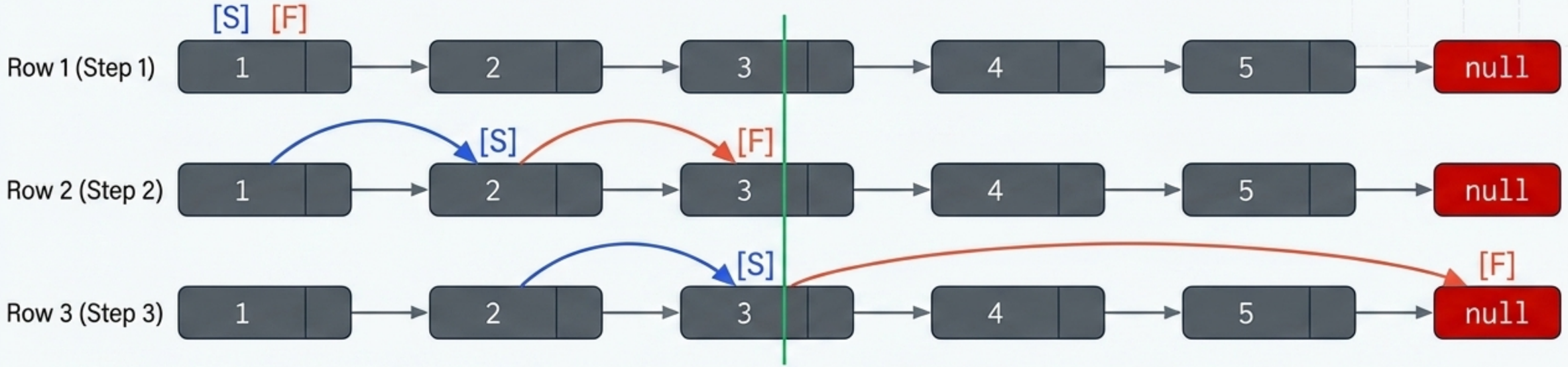


Pointers converge until they meet or a mismatch forces an early exit.

```
left = 0
right = array.length - 1

while (left < right):
    if (array[left] != array[right]):
        return False // Early stopping condition

    left = left + 1
    right = right - 1

return True
```

Playbook 2: Fast/Slow Pointers (Same Direction)

Top Panel (Visual Execution)



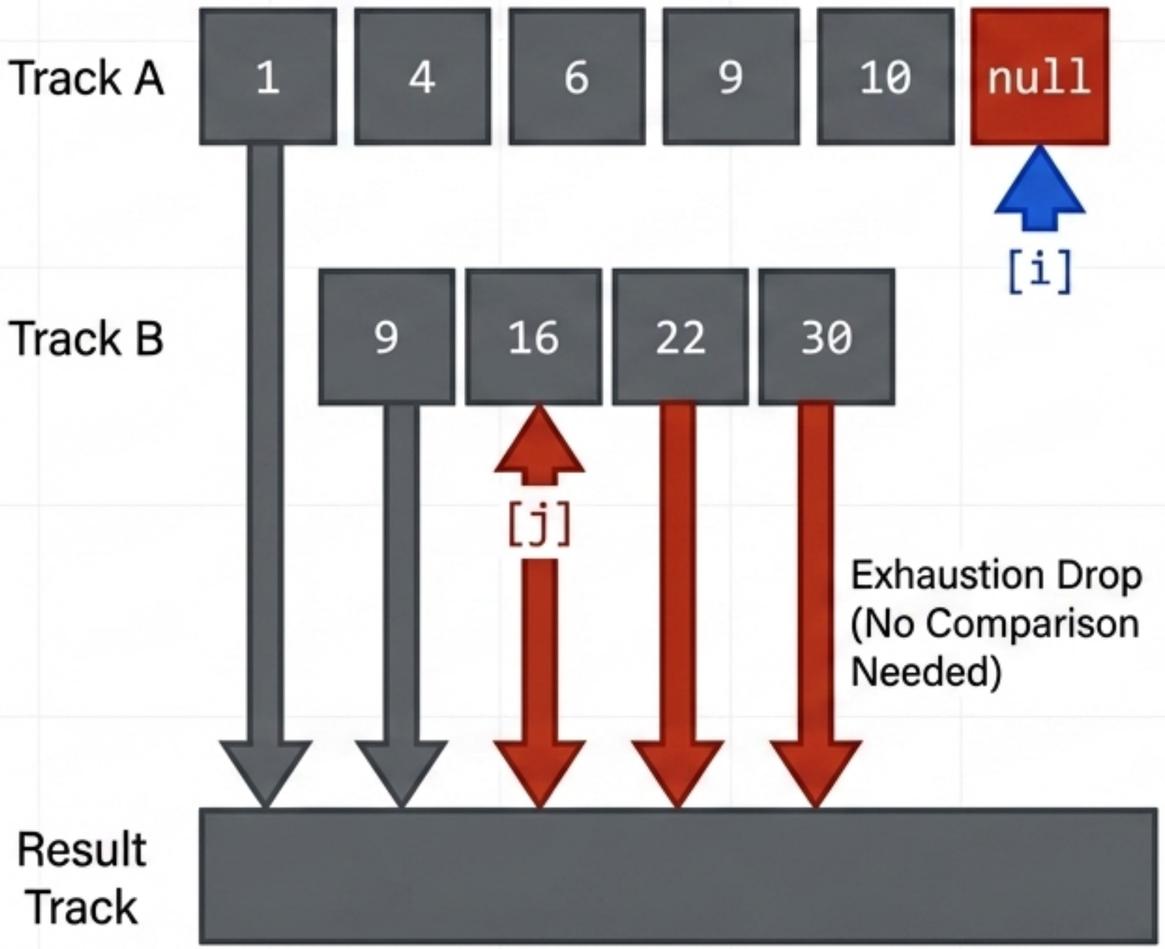
Because fast travels at 2x speed, when it hits the end, slow is mathematically guaranteed to be exactly in the middle.

Bottom Panel (Code Template)

```
slow = head
fast = head

while (fast != null and fast.next != null):
    slow = slow.next        // Moves 1 step
    fast = fast.next.next   // Moves 2 steps

    if (slow == fast):
        return True // Cycle detected
```

Playbook 3: Two Inputs (The Exhaustion Merge)

Visual Data Flow Diagram



Code Template

```
while (i < array1.length AND j < array2.length):
    if (array1[i] < array2[j]):
        result.push(array1[i]); i += 1
    else:
        result.push(array2[j]); j += 1

// Exhaustion Phase: Push remaining elements
while (i < array1.length): result.push(array1[i]); i += 1
while (j < array2.length): result.push(array2[j]); j += 1
```

The Path Forward: Day 1 Done

The 90-Day DSA Challenge Onboarding:

- [x] **Theory Absorbed:** Understand the PCS framework and Two Pointer mechanics.
- [ ] **Environment Prep:** Fork the official GitHub repository and locate the Day 1 Start File to code alongside the live sessions.
- [ ] **Foundation Check:** Review the basic Array, String, and Linked List playlists to ensure baseline familiarity before advanced pattern application.
- [ ] **Commitment:** Drop ‘Day 1 Done’ in the comments to anchor your consistency. Next up: Daily problem solving begins Monday.